

AI Hakemli Emanet

Birbirini denetleyen üç yapay zekâ hakemi ile aracısız emanet ödeme. Mimari ve akıllı kontrat yapısı.

İş veren ödemeyi akıllı kontrata kilitler. İş yapan teslimatı gönderir. Birbirinden bağımsız üç AI hakem teslimatı şartnameye göre değerlendirir ve kararını kendi cüzdanıyla zincire yazar. İki hakem aynı kararı verdiği anda para ya iş yapana gider ya da iş verene döner. Çoğunluğa ters oy veren hakemin teminatı kesilir ve haklı çıkan tarafa aktarılır.



1 · Sistem mimarisi

2 · İş akışı ve durumlar

3 · Kontrat yapısı

4 · Oylama ve teminat cezası

5 · Güvenlik modeli

6 · Sınırlar ve yol haritası

AĞ Monad testnet chainId 10143	KONTRAT AIEscrow.sol Solidity 0.8.28	HAKEM NODE'U judge.py Python · web3.py	KAYNAK github.com/ fkuyumcu/monad-blitz
--------------------------------------	--	--	---

Sistem mimarisi

Sistem dört parçadan oluşuyor: tek bir akıllı kontrat, üç bağımsız hakem node'u, iş veren ve iş yapanın kullandığı istemci, bir de salt okunur canlı ekran. Parayı tutma, oyları sayma ve ceza kesme işlerinin hepsi kontratta yapılıyor. Zincir dışında kalan tek şey, her hakemin LLM'e sorduğu soru ve verdiği kararı kendi anahtarıyla imzalaması.

ZİNCİR DIŞI • TARAFLAR



ödeme / iade + ceza payı

MONAD • ZİNCİR ÜSTÜ

AIEscrow

- İşler ve kilitli ödeme
- Hakem teminatları
- Oy kaydı ve 2/3 sayım
- Ödeme ya da iade
- Ters oya teminat cezası
- Olaylar (event log)

olaylar

Canlı ekran

dashboard • yalnızca okur

ZİNCİR DIŞI • HAKEMLER

Hakem I

cüzdan A • Claude

Hakem II

cüzdan B • GPT

Hakem III

cüzdan C • Gemini

Şekil 1 – Bileşenler. Düz oklar işlem (transaction), kesikli yeşil ok kontratın gönderdiği ödemedir.

Bileşenler

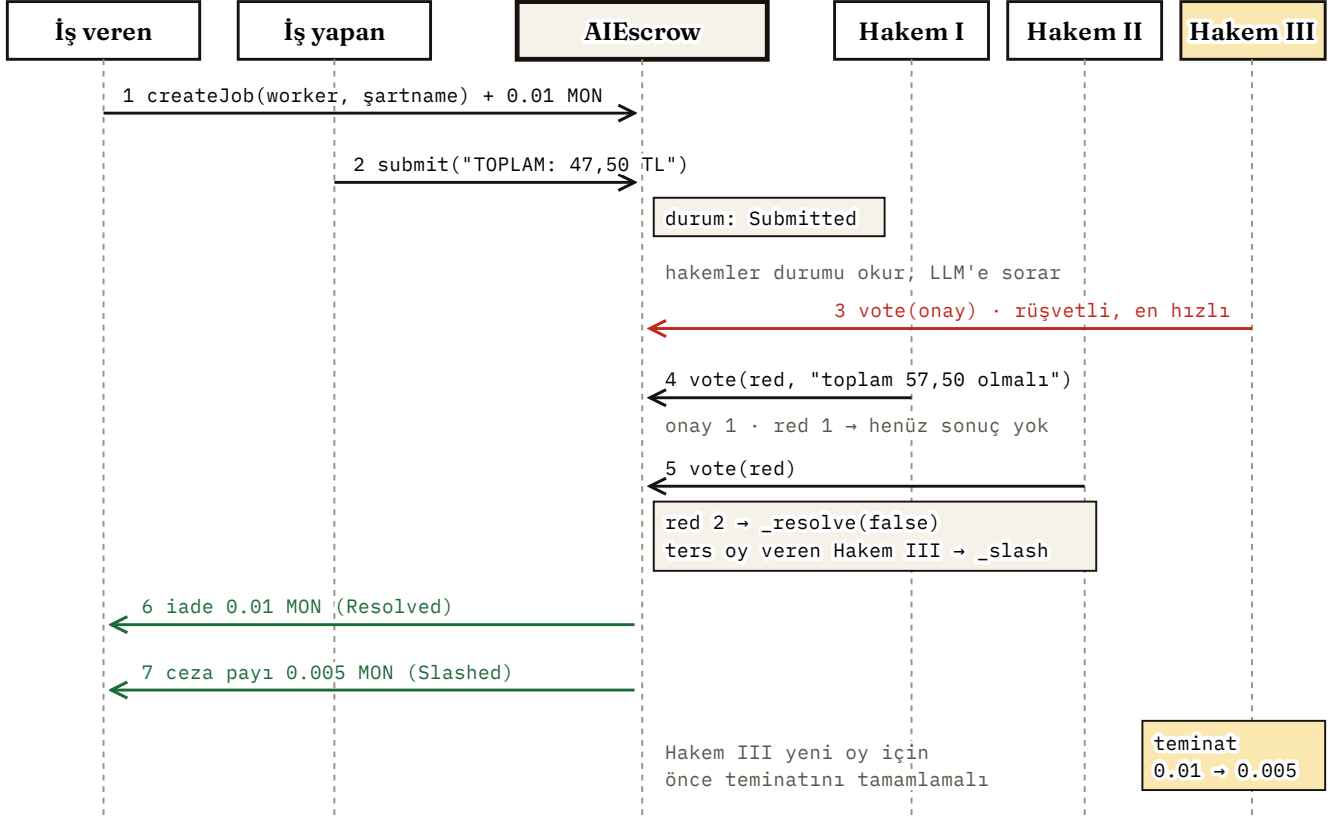
BİLEŞEN	DOSYA	GÖREVİ
Akıllı kontrat	contracts/AIEscrow.sol	Parayı emanette tutar, teminatları yönetir, oyları sayar, sonucu uygular, cezayı keser. Yöneticisi yoktur.
Hakem node'u	judge.py	Teslim edilen işleri durum sorgusuyla bulur, LLM'e sorar, kararı kendi cüzdanıyla vote olarak gönderir. Teminatı eksikse önce tamamlar.
Kurulum	deploy.py	Kontratı deploy eder. İstenirse 3 hakem cüzdanı üretip fonlar, ekran ayarını yazar.
Taraf istemcisi	demo.py	İş açar, teslimat gönderir, durum gösterir. Hazır senaryolar: doğru, yanlış, hile denemesi.
Canlı ekran	dashboard/	Olayları ve durumu okuyup hakem kartlarını, hükmü ve tutanağı gösterir. İşlem göndermez.
Ortak katman	chain.py	Bağlantı, ayar okuma ve gas limiti açıkça verilmiş işlem gönderme.

GÜVEN SINIRI

Hiçbir sunucu parayı tutmaz ve oy saymaz. Bir hakemin anahtarı çalınrsa ya da hakem satın alınsa bile sonucu tek başına değiştiremez. Bu yüzden oylamanın zincirde yapılması tasarımın vazgeçilmez kısmı.

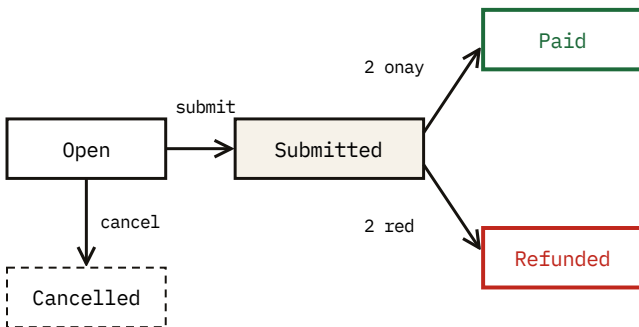
İş akışı ve durumlar

Aşağıdaki sıra diyagramı, doğru bir teslimatın ve rüşvetli bir hakemin olduğu senaryoyu gösteriyor. Rüşvetli hakem LLM'e sormadığı için en hızlıdır ve ilk oyu verir. Diğer iki hakem ters yönde oy verince iş iade edilir ve rüşvetçinin teminatı kesilir.



Şekil 2 – Yanlış teslimat ve rüşvetli hakem senaryosu (demo.py run bad). Tutarlar deploy varsayılanları.

İş durumları



Şekil 3 – Durum makinesi. Paid ve Refunded son durumlardır; sonrasında yalnızca geç oy ve ceza olabilir.

Hakem döngüsü (judge.py)

```

her 1,5 sn:
    n = jobCount()
    her iş için:
        Submitted mı ve oy vermedim mi?
        → LLM(şartname, teslimat)
        rastgele etiketli veri bloğu,
        çıktı: {"approve", "reason"}
        → teminat eksikse stake()
        → vote(id, approve, "[model] gerekçe")
  
```

Hakemler olay yerine durum sorgusu kullanıyor. Böylece public RPC'lerin eth_getLogs aralık sınırlarına takılmıyorlar ve yeniden başlatıldıklarında bekleyen işleri kaçırmıyorlar.

Kontrat yapısı

Tek kontrat, dış bağımlılık yok. Derleyici solc 0.8.28, optimizier 200 run, evmVersion: cancun. Kontratın yöneticisi (owner) yok. Hakemler ve ekonomik parametreler deploy anında sabitleniyor.

Depolama

ALAN	TİP
JUDGE_COUNT / QUORUM	sabit 3 / 2
judgeStake	immutable · asgari teminat
slashAmount	immutable · ters oy cezası
judges	address[3]
isJudge	address ⇒ bool
stakeOf	address ⇒ uint256
_jobs	Job[] · id = indeks
voteOf	id ⇒ hakem ⇒ 0 1 2

Job

```
struct Job {
    address client;        // iş veren
    address worker;        // iş yapan
    uint256 amount;        // kilitli ödeme
    Status status;        // Open ... Cancelled
    uint8  approvals;
    uint8  rejections;
    string spec;           // şartname
    string deliverable;    // teslimat
}
```

Fonksiyonlar

İMZA	ÇAĞIRAN	KOŞUL	ETKİ · OLAY
constructor(address[3], stake, slash)	deployer	adresler geçerli ve tekrarsız, slash ≤ stake	hakemleri sabitler
stake() payable	hakem	isJudge	teminat artar · JudgeStaked
withdrawStake(amount)	hakem	amount ≤ teminat	geri ödeme · JudgeWithdrew
createJob(worker, spec) payable	herkes	value > 0	iş açılır · JobCreated
cancel(id)	iş veren	durum Open	iade · Cancelled
submit(id, deliverable)	iş yapan	durum Open	Submitted · Submitted
vote(id, approve, reason)	hakem	teminat yeterli, teslim edilmiş, ilk oy	Voted; çoğunlukta Resolved, ters oyda Slashed
jobCount · getJob · getJudges	view	—	okuma

Olaylar ve hatalar

```
JobCreated(id, client, worker, amount, spec)
Submitted (id, worker, deliverable)
Voted (id, judge, approve, reason)
Resolved (id, approved, to, amount)
Slashed (id, judge, to, amount)
Cancelled (id)
JudgeStaked / JudgeWithdrew
```

```
NotJudge · NotClient · NotWorker
BadStatus 0x5c975bda
AlreadyVoted 0x7c9a1cf9
StakeTooLow 0x1cc3b37b
ZeroAmount · BadJudges
TransferFailed
```

Oylama ve teminat cezası

Kararın özü vote fonksiyonunda. Oy kaydedilir, sayaç artar ve iş hâlâ açıksa çoğunluk kontrol edilir. İş zaten sonuçlanmışsa gelen oy yine kaydedilir, ama sonuçla uyuşmuyorsa ceza kesilir. Böylece bir hakem son sırada oy verip riskten kaçamaz.

```
function vote(id, approve, reason):
  require isJudge(msg.sender) and stakeOf[msg.sender] ≥ judgeStake
  require job.status ∈ {Submitted, Paid, Refunded} and voteOf[id][msg.sender] == 0
  voteOf[id][msg.sender] = approve ? 1 : 2 ; approve ? approvals++ : rejections++
  emit Voted

  if job.status == Submitted:
    if approvals ≥ 2: _resolve(id, true) // ödeme iş yapana
    if rejections ≥ 2: _resolve(id, false) // iade iş verene
  else if approve ≠ (job.status == Paid):
    _slash(id, msg.sender) // geç gelen ters oy

_resolve(id, approved):
  status = approved ? Paid : Refunded // önce durum (reentrancy'ye karşı)
  o ana kadar ters oy veren her hakem için _slash
  parayı gönder → emit Resolved

_slash(id, judge):
  kesinti = min(stakeOf[judge], slashAmount); stakeOf[judge] -= kesinti
  kesintiyi kazanan tarafa gönder → emit Slashed
```

Ekonomik parametreler (deploy varsayıları)

PARAMETRE	DEĞER	ANLAMI
judgeStake	0.01 MON	Oy verebilmek için gereken asgari teminat. Teminatı kesilen hakem, eksiği tamamlamadan tekrar oy veremez.
slashAmount	0.005 MON	Ters oy başına kesinti. Kesinti hazineye değil, sonuçta haklı çıkan tarafa gider.
iş tutarı	0.01 MON	demo.py varsayıları.

Değişmezler

#	KURAL	NASIL KORUNUYOR
1	Bir iş en fazla bir kez sonuçlanır.	status kontrolü, test
2	Bir hakem bir işe en fazla bir kez oy verir.	voteOf, AlreadyVoted
3	Kontrat bakiyesi = açık işlerin tutarları + tüm teminatlar.	test (yalnızca teminatlar kalır)
4	Ceza, hakemin teminatını aşmaz.	min() ve slash ≤ stake
5	Durum değişiklikleri para gönderiminden önce yapılır.	checks-effects-interactions

test_escrow.py: 7/7 geçiyor. Kapsam: onayda ödeme, redde iade, önce gelen ve geç gelen ters oya ceza, uyumlu geç oya ceza yok, yetki ve durum korumaları, teminatsız hakem, iptal.

Güvenlik modeli

TEHDİT	SAVUNMA
Tek hakemin satın alınması ya da anahtarının çalınması	Sonuç için 2/3 aynı oy gerekiyor. Ters oy veren teminat kaybediyor, kesinti haklı tarafa gidiyor. Tek başına sonucu değiştiremez, üstelik zarar eder.
Parayı tutan aracının kaçması ya da ödemeyi geciktirmesi	Aracı yok. Para kontratta, çıkış yolları yalnızca kurallarla tanımlı (sonuç, iptal, ceza).
Teslimat içine gömülü talimat (prompt injection)	Şartname ve teslimat, her çağrıda yeniden üretilen rastgele etiketler (<TESLIMAT_9f3a...>) arasında veri olarak veriliyor. Sistem mesajı, hakeme talimat vermeye çalışan teslimatı hile sayıp reddetmesini söylüyor. Çıktı katı JSON, ayrıştırılmazsa oy verilmiyor.
Hakemlerin aynı hatayı yapması	Her hakem farklı bir sağlayıcı kullanıyor (Claude, GPT, Gemini) ve farklı bir kişi tarafından çalıştırılabilir (deploy.py --judges).
Geç oyla riskten kaçma	Sonuçtan sonra gelen ters oy da cezalandırılıyor.
Öznel işlerde tartışmalı karar	Kapsam dışı. Demo görevi nesnel (fişten toplam tutar, doğru cevap 57,50 TL).

NEDEN BLOCKCHAIN?

Birbirine güvenmeyen iki taraf var ve parayı tutacak güvenilir bir aracı yok. Kararı tek bir hakeme ya da tek bir platforma bırakmak, onu tek güven noktası yapar. Kontrat hem emanetçi hem de sayım görevlisi. Oylar ve cezalar herkesin denetleyebileceği şekilde zincirde duruyor. Üç hakemi aynı kişi çalıştırırsa bu güvence ortadan kalkar. Bu yüzden hakemlerin farklı operatörlere dağıtılması önemli.

Monad'a özgü notlar

Monad gas'ı kullanılan miktara göre değil, verilen gas limitine göre kesiyor. chain.send() bu yüzden her işlemde gas'ı tahmin edip %15 pay ekleyerek açıkça veriyor. ~0,4 saniyelik blok süresi sayesinde oylar ve sonuç ekrana neredeyse anında düşüyor. Düşük işlem ücreti, fikrin küçük tutarlı işlere ve ajanlar arası görevlere ölçeklenmesini mümkün kılıyor.

Sınırlar ve yol haritası

Bu sürüm tek günlük bir hackathon için yazıldı. Aşağıdaki sınırlar bilinçli olarak bırakıldı. Her birinin bilinen bir çözümü var.

SINIR	RİSK	ÇÖZÜM
Sabit 3 hakem	Hakem kümesi deploy edenin seçimine bağlı.	Açık kayıt ve her iş için rastgele seçim (VRF ya da commit-reveal ile blockhash).
Oylar açık	Sonraki hakem öncekinin oyunu görüp kopyalayabilir.	Commit-reveal oylama.
Zaman aşımı yok	Hakemler oy vermezse iş askıda kalır.	Deadline sonrası iade ve hakem değişimi.
Push ödeme	ETH kabul etmeyen bir taraf sonucu kilitleyebilir (DoS).	Pull-payment: <code>withdraw()</code> .
Teminat çekme serbest	Hakem kötü oydan önce teminatını çekip cezadan kaçabilir.	Açık oyları olan hakemin teminatını kilitlemek.
Teslimat zincirde düz metin	Maliyet ve gizlilik.	IPFS CID ve zincirde yalnızca hash.

Sonraki aşama (Monad Metropolis, 13 Ekim)

#	ADIM
1	Açık hakem havuzu, teminat ağırlıklı rastgele seçim, commit-reveal oylama.
2	Zincirde hakem itibarı: çoğunlukla uyum oranı, seçilme olasılığını etkilesin.
3	Deadline, itiraz turu (5 hakemli ikinci tur), pull-payment.
4	Konumlandırma: ajanlar arası mikro görevler. Burada komisyonlu platformlar ekonomik değil, düşük ücretli zincir fark yaratıyor.
5	Foundry'ye geçiş, fuzz ve invariant testleri, bağımsız güvenlik incelemesi.

Çalıştırma özeti

```
pip install "web3[tester]" requests && python test_escrow.py
python deploy.py --gen-judges && python demo.py setup
python judge.py --env judge1.env # x3, biri --corrupt
cd dashboard && python -m http.server 8080
python demo.py run good | bad | inject
```